Project - High Level Design

on

AGRICULTURE SUPPORT

TRIAGE AGENT

Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Deepak Patro	EN22CS301316
2.	Bhoomika Patidar	EN22CS301270
3.	Charu Nariya	EN22CS301290
4.	Dev Kumar Das	EN22CS301321
5.	Avichal Mishra	EN22CS301243

Group Name:07D3

Project Number:AAI-31

Industry Mentor Name:

University Mentor Name:Prof. Maya Yadav

Academic Year:2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1.INTRODUCTION

The **AI Agriculture Support System** is an intelligent decision-support platform designed to help farmers improve productivity and make informed agricultural decisions.

The system uses **Agentic AI architecture**, where multiple specialized AI agents collaborate to solve different types of agricultural problems. It integrates **Retrieval Augmented Generation (RAG)** with Large Language Models (LLMs) to ensure that responses are not only intelligent but also based on real agricultural data.

This system provides assistance in:

- Crop selection
- Disease identification
- Government policy awareness

The goal is to make agriculture more efficient, data-driven, and accessible to farmers.

1.1 SCOPE OF THE DOCUMENT

This document describes the **High Level Design (HLD)** of the AI Agriculture Support System. It provides an overview of the system architecture, main components, and how they interact with each other.

The scope includes:

- Overall system structure and layers
- Key components like UI, AI agents, and RAG system
- Basic process and information flow
- External APIs and services used

It does not include detailed code implementation or low-level design.

1.2 INTENDED AUDIENCE

This document is intended for:

- Developers who will build and maintain the system
- System architects who design the overall structure
- Mentors and project reviewers for evaluation

1.3 SYSTEM OVERVIEW

The **AI Agriculture Support System** is a multi-agent AI-based platform designed to assist farmers in making informed agricultural decisions. The system provides guidance in key areas such as crop selection, disease identification, and government schemes.

It works by allowing users to enter their queries through a simple user interface. The system then processes the query using a **CrewAI-based multi-agent architecture**, where each agent is responsible for a specific task.

The system includes three main agents:

- **Crop Advisor Agent** for crop recommendations and farming practices
- **Disease Expert Agent** for identifying crop diseases and suggesting treatments
- **Policy Advisor Agent** for providing information about government schemes

To ensure accurate and reliable responses, the system uses **Retrieval Augmented Generation (RAG)**, which fetches relevant agricultural data from a knowledge base and combines it with a Large Language Model.

Overall, the system acts as an intelligent assistant that provides fast, accurate, and useful agricultural advice to farmers in an easy-to-use manner.

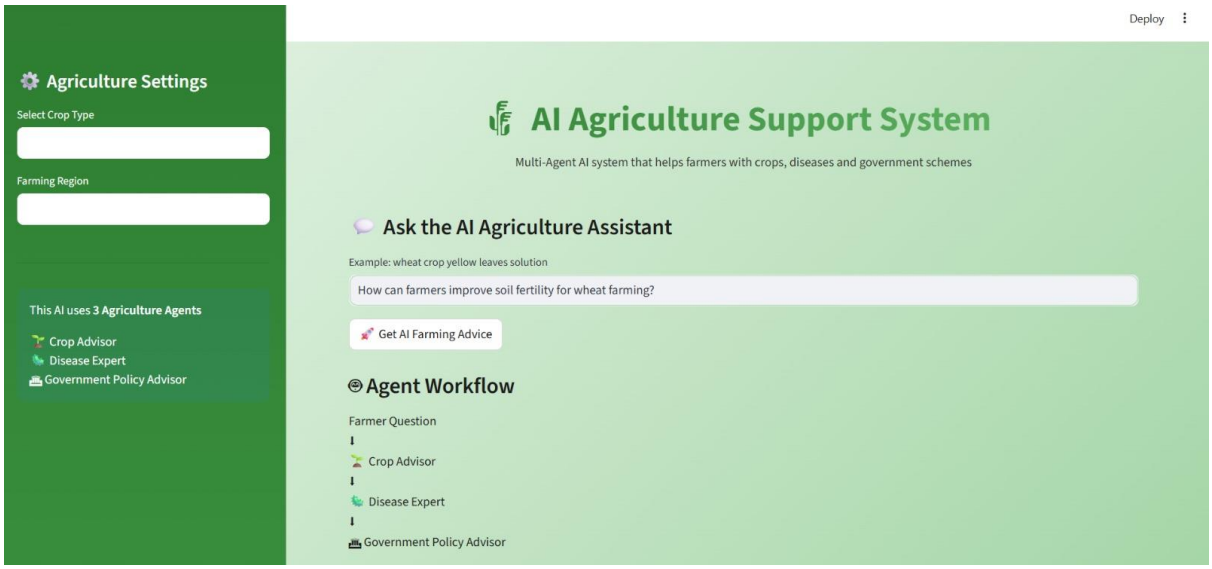
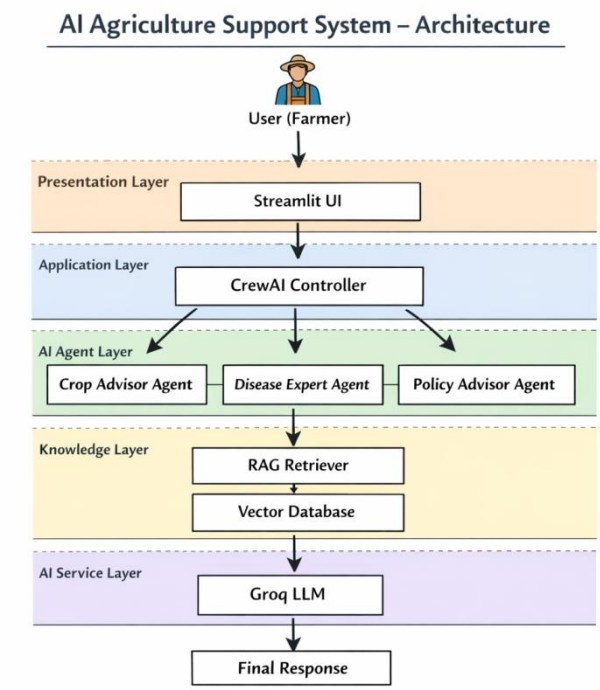


Figure: Main Interface of AI Agriculture Support System

2.SYSTEM DESIGN

The AI Agriculture Support System is designed using a **layered and modular architecture** to ensure flexibility, scalability, and efficient processing of user queries. Each layer in the system has a specific responsibility and communicates with other layers to deliver accurate results.

The design focuses on separating concerns such as user interaction, business logic, AI processing, and data handling, which improves maintainability and system performance.



2.1 APPLICATION DESIGN

The application follows a **modular design approach** where different functionalities are divided into independent modules. This makes the system easy to understand, develop, and maintain.

Key modules include:

- **App.py:** Handles user interaction using Streamlit UI
- **Agents.py:** Defines multiple AI agents with specific roles
- **Tasks.py:** Contains tasks assigned to each agent
- **Crew_setup.py:** Controls execution and coordination of agents
- **Rag_setup.py:** Implements Retrieval Augmented Generation
- **Data_collector.py:** Collects and processes agricultural data

This design ensures:

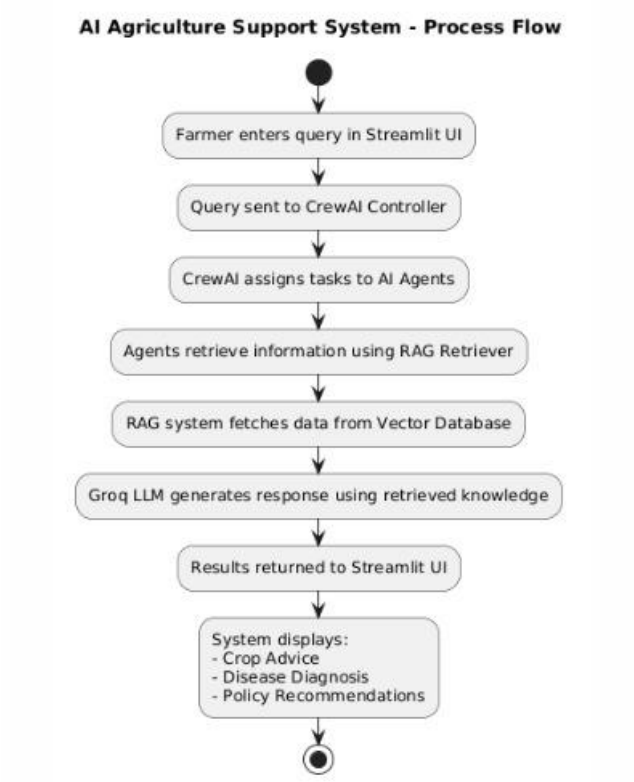
- Easy debugging
- Reusability of components
- Scalability for adding new features

2.2 PROCESS FLOW

The system processes user queries through a structured workflow:

1. The user enters a query related to agriculture
2. The query is received by the Streamlit interface
3. It is forwarded to the CrewAI controller
4. The controller analyzes the query type

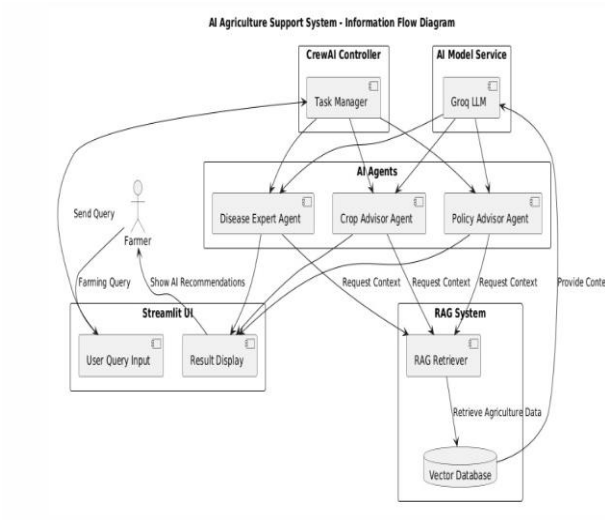
5. Based on the query, the appropriate agent is selected
6. The selected agent interacts with the RAG system
7. Relevant data is retrieved from the vector database
8. The LLM combines the query and retrieved data
9. A meaningful response is generated
10. The result is displayed to the user



2.3 INFORMATION FLOW

The system follows a clear and organized flow of information:

- User provides input through UI
- Input is passed to the controller
- Controller sends it to the appropriate agent
- Agent requests data from RAG
- RAG retrieves information from the vector database
- Data is sent to the LLM
- LLM generates the final response
- Response is displayed back to the user



2.4 COMPONENT DESIGN

The system consists of several key components, each with a defined role:

- **Streamlit UI**
  - Provides user-friendly interface
  - Accepts queries and displays responses
- **CrewAI Controller**
  - Acts as the central coordinator
  - Assigns tasks to agents
  - Manages workflow
- **Crop Advisor Agent**
  - Recommends suitable crops
  - Suggests fertilizers and irrigation methods
- **Disease Expert Agent**
  - Identifies plant diseases
  - Suggests treatments and prevention
- **Policy Advisor Agent**
  - Provides information about government schemes
  - Helps farmers access benefits
- **RAG Retriever**
  - Retrieves relevant agricultural data
  - Improves response accuracy
- **Vector Database**
  - Stores agricultural knowledge as embeddings
  - Enables fast data retrieval

2.5 KEY DESIGN CONSIDERATION

The system is designed keeping the following factors in mind:

- **Accuracy:** Ensured using RAG-based retrieval
- **Scalability:** New agents can be added easily

- **Usability:** Simple interface for non-technical users
- **Maintainability:** Modular code structure
- **Flexibility:** Easy integration with new APIs or datasets
- **Performance:** Optimized query handling and fast responses

2.6 API CATALOGUE

API / Tool	Purpose
Groq API	Provides LLM-based response generation
Streamlit	Builds user interface
LangChain	Enables RAG functionality
CrewAI	Manages multi-agent system

3. DATA DESIGN

The data design of the AI Agriculture Support System focuses on efficient storage, retrieval, and management of agricultural information. The system uses a **vector database and RAG-based approach** to ensure fast and accurate access to relevant data.

3.1 DATA MODEL

The system uses a **document-based data model** where agricultural knowledge is stored in the form of text documents.

Types of data include:

- Crop information (types, seasons, requirements)
- Disease data (symptoms, causes, treatments)
- Government schemes and policies
- General farming practices

These documents are converted into **vector embeddings** and stored in a vector database. This allows semantic search instead of keyword-based search, improving accuracy.

3.2 DATA ACCESS MECHANISM

The system uses **Retrieval Augmented Generation (RAG)** for accessing data.

Steps involved:

1. User query is converted into an embedding
2. The system searches the vector database
3. Relevant documents are retrieved
4. Retrieved data is passed to the LLM
5. LLM generates a context-aware response

This mechanism ensures:

- Fast data retrieval
- Relevant and accurate answers
- Better understanding of user queries

3.3 DATA RETENTION POLICIES

The system follows structured data retention practices to ensure data accuracy, reliability, and efficient storage management.

- **Long-Term Storage:**
  Agricultural knowledge such as crop data, disease information, and government schemes is stored for long-term use to support continuous system functionality.
- **Regular Updates:**
  Data is periodically updated to reflect the latest agricultural practices, weather patterns, and government policies.
- **Removal of Outdated Data:**
  Old or irrelevant data is identified and either updated or removed to maintain the quality of responses.
- **Version Control:**
  Important datasets may maintain versions to track changes and ensure consistency over time.
- **No Personal Data Storage:**
  The system does not store sensitive personal information of users, ensuring privacy and security.
- **Data Consistency:**
  Mechanisms are used to ensure that all stored data remains consistent and valid across the system.
- **Backup and Recovery:**
  Data backups can be maintained to prevent data loss and support recovery in case of system failure.

The system supports flexible and scalable data migration to handle updates and integration of new data sources.

3.4 DATA MIGRATION

The system supports flexible and scalable data migration to handle updates and integration of new data sources.

- **Adding New Data Sources:**
  New agricultural datasets (e.g., new crops, diseases, or policies) can be easily added to the system.
- **Data Updating:**
  Existing data can be modified or replaced without affecting system performance.
-
-

- **Embedding Regeneration:**
  During migration, data is reprocessed into vector embeddings to maintain compatibility with the vector database.
- **Seamless Integration:**
  Migration is designed to occur without interrupting the system's operation.
- **Scalability Support:**
  The system can handle increasing data volume as more information is added over time.
- **Data Validation:**
  New data is validated before being added to ensure accuracy and reliability.
- **Format Compatibility:**
  The system supports different data formats (text, documents, datasets) for easy integration.

4. INTERFACES

The system interacts with users and external services through well-defined interfaces.

1. User Interface

- Built using **Streamlit**
- Allows users (farmers) to enter queries
- Displays responses in a simple and readable format

2. API Interfaces

- **Groq API:** Used for generating responses through LLM
- **LangChain:** Used for implementing RAG-based retrieval
- **CrewAI:** Used for managing multi-agent communication

3. Internal Interfaces

- Communication between controller and agents
- Interaction between agents and RAG system
- Data exchange between RAG and vector database

These interfaces ensure smooth communication across all parts of the system.

5.STATE AND SESSION MANAGEMENT

The system manages user interactions efficiently using session handling.

- **Session Handling:**
  Each user interaction is managed as a session in Streamlit
- **Temporary State Storage:**
  Stores user queries and responses during a session
-

- **Context Maintenance:**
  Maintains short-term context to provide better responses
- **Stateless Backend:**
  The backend remains mostly stateless, improving scalability
- **Session Isolation:**
  Each user session is independent to avoid data conflicts

This ensures smooth and personalized interaction for users.

6.CACHING

Caching is used to improve system performance and reduce response time.

- **Query Caching:**
  Frequently asked queries are stored and reused
- **Result Caching:**
  Previously generated responses can be reused
- **RAG Data Caching:**
  Retrieved documents can be temporarily cached
- **Performance Improvement:**
  Reduces repeated computation and API calls
- **Efficient Resource Usage:**
  Minimizes load on the system and APIs

Caching helps the system respond faster and work more efficiently.

7.NON-FUNCTIONAL REQUIREMENT

Non-functional requirements define the quality and performance standards of the system. They ensure that the system is reliable, efficient, and user-friendly.

- **Scalability:**
  The system should handle increasing users and data without performance issues. New agents and features can be added easily.
- **Reliability:**
  The system should provide consistent and accurate results with minimal failures.
- **Usability:**
  The interface should be simple and easy to use, especially for non-technical users like farmers.
- **Maintainability:**
  The modular design allows easy updates, debugging, and improvements.
- **Availability:**
  The system should be accessible whenever users need it, with minimal downtime.

7.1 SECURITY ASPECTS

Security is important to protect system data and ensure safe usage.

- **API Key Protection:**
  API keys (like Groq API) are securely stored and not exposed publicly.
- **Data Privacy:**
  The system does not store sensitive personal user data.
- **Secure Communication:**
  Data exchange between components and APIs is handled securely.
- **Access Control:**
  Only authorized components can access certain services or data.
- **Protection from Misuse:**
  Basic validation is applied to user inputs to prevent misuse or errors.

7.2 PERFORMANCE ASPECTS

Performance ensures that the system responds quickly and efficiently.

- **Fast Response Time:**
  The system should generate responses quickly using optimized LLM and RAG.
- **Efficient Data Retrieval:**
  Vector database enables fast and relevant data search.
- **Caching:**
  Frequently used queries and data are cached to reduce processing time.
- **Optimized Processing:**
  Agents handle only relevant tasks, improving efficiency.
- **Low Latency:**
  The system minimizes delays in query processing and response generation.

8.REFRENCES

- CrewAI Documentation
- LangChain Documentation
- Groq API Documentation
- Streamlit Documentation